

Programação Orientada a Objetos

Capítulo 3 — Os 4 Princípios da Orientação a Objetos

Aula 03

O que veremos hoje?

- 1 Os 4 princípios fundamentais da OO
- 2 Abstração: do mundo real ao computacional
- 3 Encapsulamento: segurança e sigilo
- 4 Herança: reutilização e produtividade
- 5 Polimorfismo: várias formas, mesmo método
- 6 **Dinâmicas & Atividades práticas**



3.1 Princípios da OO

Abstração, Encapsulamento, Herança e Polimorfismo

Os 4 Princípios Fundamentais da OO



Abstração

Identificar objetos, atributos e métodos relevantes do mundo real



Encapsulamento

Esconder detalhes internos, controlar acesso público/privado



Herança

Reutilizar características de classes existentes em novas classes



Polimorfismo

Mesmo método com comportamentos diferentes conforme o contexto

Por que os 4 princípios são essenciais?

- Um projeto que não siga os 4 princípios não é considerado 100% orientado a objetos;
- Desde a análise do problema até as últimas linhas de código, o pensamento OO deve ser mantido;
- Os princípios garantem segurança no código + reutilização e reaproveitamento eficiente;
- A implementação varia entre linguagens, mas o conceito teórico é universal;
- A linguagem deve oferecer suporte a todos os 4 para ser considerada orientada a objetos;
- No início parece difícil, mas com prática se torna natural.



3.2 Abstração

Isolar objetos relevantes do mundo real para o sistema

O que é Abstração?

- Talvez o princípio mais importante: responsável por toda a maneira de pensar ao preparar o sistema;
- Abstração = isolar um objeto de reflexão dos fatores que normalmente lhe estão relacionados na realidade;
- Transformar objetos do mundo real em objetos utilizáveis computacionalmente;
- Identificar quais objetos são relevantes ao projeto e quais atributos e métodos cada um terá;
- Nem tudo do cenário real entra no sistema — apenas o que é pertinente às regras de negócio.

Abstração na prática: oficina mecânica

- Objetos encontrados: carro, pneu, lâmpada, mesa, ferramenta, cadeira, mecânico, peças...
- Lâmpada: irrelevante para as regras de negócio — não precisa entrar no sistema.
- Carro: muito importante — é o objeto central do processo da oficina.
- Pneu: depende do contexto — se a oficina faz serviço de borracharia, é relevante; se não, pode ser dispensado.
- Caixa: objeto com ação 'fechamento de caixa' — encerra a movimentação financeira do dia.
- A abstração define **O QUÊ** entra no sistema e **COM QUAIS** propriedades e ações.

Dinâmica 1 — Abstraindo um Cenário Real

- Em grupos de 3–4, vocês são analistas contratados para criar o sistema de um HOSPITAL;
- Listem todos os objetos que vocês encontram num hospital (sem filtro, anotem tudo);
- Agora, apliquem a abstração: quais objetos são RELEVANTES para um sistema de gestão hospitalar?
- Para cada objeto relevante, definam: 2 atributos + 1 método;
- Quais objetos vocês DESCARTARAM e por quê?
- Tempo: 8 minutos. Apresentação: 2 minutos por grupo.

Exemplos de resposta — Dinâmica 1

- Objetos encontrados: médico, enfermeiro, paciente, maca, bisturi, prontuário, cadeira, TV, receita, exame...
- Relevantes: Médico (CRM, especialidade) → `atenderPaciente()`; Paciente (nome, convênio) → `agendarConsulta()`.
- Prontuário (data, diagnóstico) → `registrarEvolução()`; Exame (tipo, resultado) → `emitirLaudo()`.
- Descartados: cadeira, TV, bisturi — existem no hospital mas não fazem parte das regras de negócio do sistema.
- Chave: a abstração filtra o que é relevante e ignora o que não impacta os processos do *software*.



3.3 Encapsulamento

Esconder detalhes internos e proteger o código

O que é Encapsulamento?

- Principal conceito para segmentação e segurança do código fonte;
- Consiste em esconder detalhes internos de métodos e atributos que não devem ser compartilhados;
- Evita que um método chame outro indevidamente ou que classes externas alterem dados sensíveis;
- Permite profissionais especializados apenas em suas classes, sem ver o sistema inteiro;
- Mantém informações e procedimentos da empresa com maior sigilo.

Público *vs.* Privado

- Público: métodos e atributos acessíveis a partir de outras classes — tudo que precisa ser compartilhado;
- Privado: métodos e atributos que trabalham somente em procedimentos internos da classe;
- Regra: sempre que uma informação não for necessária externamente, declare como privada;
- Exemplo: método 'fechamento de caixa' pode ser privado — o resultado é passado por mensagem, mas o processo interno fica oculto;
- Encapsular = proteger + organizar + simplificar a interface entre classes.

Dinâmica 2 — O que é Público? O que é Privado?

- Em duplas, considerem a classe 'ContaBancaria' com os seguintes membros:
 - Atributos: `saldo`, `titular`, `senha`, `historicoTransacoes`, `limiteCredito`;
 - Métodos: `depositar()`, `sacar()`, `consultarSaldo()`, `validarSenha()`, `calcularJuros()`;
 - Classifiquem cada atributo e método como **PÚBLICO** ou **PRIVADO**. Justifiquem;
 - Pergunta extra: o que poderia acontecer se 'saldo' fosse público?
- Tempo: 5 minutos.

Respostas possíveis — Dinâmica 2

- Público: `depositar()`, `sacar()`, `consultarSaldo()` — são as ações que outros objetos precisam acessar;
- Privado: `saldo`, `senha`, `historicoTransacoes`, `validarSenha()`, `calcularJuros()` — detalhes internos sensíveis;
- `limiteCredito`: pode ser privado com acesso via método público `getLimite()` (padrão *getter*);
- `titular`: pode ser público (leitura) mas protegido contra alteração externa;
- Se '`saldo`' fosse público: qualquer classe poderia alterar o valor diretamente, sem passar por `sacar()` ou `depositar()`, quebrando regras de negócio e gerando inconsistência.



3.4 Herança

Reutilizar características de classes existentes

O que é Herança?

- Característica mais marcante para agilidade no desenvolvimento e reaproveitamento de código;
- Permite usar características de uma classe existente para criar novas classes semelhantes;
- Classe base (superclasse/classe pai): fornece as características que serão herdadas;
- Classe derivada (subclasse/classe filha): recebe tudo da base e pode acrescentar novidades;
- Resultado: melhor estruturação, menos duplicação, manutenção facilitada.

Herança na prática: Pessoa → Cliente / Funcionário

- Classe base 'Pessoa': atributos `nome`, `RG`, `CPF`, `endereço` — comuns a todos;
- Classe derivada 'Cliente': herda tudo de Pessoa + adiciona `dataCadastro`, `preferencias`;
- Classe derivada 'Funcionário': herda tudo de Pessoa + adiciona `PIS`, `cargo`, `salario`;
- Sem herança: teríamos que reescrever `nome`, `RG`, `CPF` e `endereço` em cada classe separadamente;
- Com herança: alterou algo na classe Pessoa → todas as derivadas refletem automaticamente;
- Favorece reutilização de código já existente e aumenta produtividade.

Dinâmica 3 — Construindo Hierarquias de Herança

- Em duplas, criem uma hierarquia de herança para um **SISTEMA DE STREAMING DE VÍDEO**:
 - Definam 1 classe base com pelo menos 4 atributos comuns;
 - Criem 3 classes derivadas que herdem da base e adicionem atributos/métodos próprios;
 - Exemplo de partida: Classe base 'Conteudo' → Derivadas: ?
 - Desenhem a hierarquia no papel mostrando o que cada classe herda e o que adiciona.
- Tempo: 7 minutos.

Exemplo de resposta — Dinâmica 3

- Classe base 'Conteudo': `titulo`, `duracao`, `genero`, `classificacaoEtaria`. Método: `reproduzir()`;
- Derivada 'Filme': herda tudo + `diretor`, `elenco`. Método: `exibirCreditos()`;
- Derivada 'Serie': herda tudo + `temporada`, `episodio`. Método: `proximoEpisodio()`;
- Derivada 'Documentario': herda tudo + `tema`, `fontes`. Método: `exibirBibliografia()`;
- Vantagem: se adicionar '`idioma`' em Conteudo, todas as 3 derivadas ganham automaticamente.



3.5 Polimorfismo

Mesmo nome, comportamentos diferentes conforme o contexto

O que é Polimorfismo?

- Do grego: 'que pode ter várias formas'. Um método pode adotar formas diferentes conforme o contexto.
- Não é o objeto que muda — são as ações que diferem quando mudamos a forma de nos referir a ele;
- Depende de fatores como: quantidade de parâmetros, tipo de parâmetros, classe que implementa;
- Classes derivadas podem ter métodos com o mesmo nome da base, mas com comportamento particular;
- É uma forma de controlar, de maneira generalista, os métodos na classe base com particularidades nas derivadas.

Polimorfismo na prática: bônus por cargo

- Classe base 'Funcionario': método `bonusAnual()` — definido genericamente;
- Derivada 'Coordenador': `bonusAnual()` retorna salário $\times$ 7%;
- Derivada 'Gerente': `bonusAnual()` retorna salário $\times$ 10%;
- Derivada 'Diretor': `bonusAnual()` retorna salário $\times$ 12%;
- Mesmo nome de método, mas cada classe executa de forma diferente — isso é polimorfismo;
- Também pode ocorrer na mesma classe: métodos com mesmo nome mas parâmetros diferentes (sobrecarga).

Dinâmica 4 — Identificando Polimorfismo

- Individualmente, considere o cenário: sistema de transporte com veículos;
- Classe base 'Veiculo' com método `calcularTarifa()`;
- Classes derivadas: Onibus, Taxi, Mototaxi;
- Defina como `calcularTarifa()` seria **DIFERENTE** em cada classe derivada;
- Bônus: crie um exemplo de sobrecarga — mesmo método, parâmetros diferentes na mesma classe;
- Tempo: 5 minutos.

Respostas possíveis — Dinâmica 4

- Onibus: `calcularTarifa()` → retorna valor fixo (R\$ 4,50), independente da distância;
- Taxi: `calcularTarifa()` → `bandeirada + (km × valorPorKm) + tempoParado`;
- Mototaxi: `calcularTarifa()` → `distância × valorPorKm` (sem bandeirada, tarifa mais simples);
- Sobrecarga: `calcularTarifa()` vs. `calcularTarifa(desconto)` — com 1 parâmetro, aplica percentual de desconto;
- Mesmo método '`calcularTarifa`', 3 comportamentos totalmente diferentes = polimorfismo.

Dinâmica 5 — Integradora: Os 4 Princípios Juntos

- Em grupos de 4, projetem um mini-sistema para uma LOCADORA DE VEÍCULOS.
- Apliquem os 4 princípios e identifiquem para cada um:
 - **ABSTRAÇÃO**: quais objetos, atributos e métodos são relevantes?
 - **ENCAPSULAMENTO**: o que deve ser público e o que deve ser privado?
 - **HERANÇA**: existe uma classe base que pode gerar derivadas?
 - **POLIMORFISMO**: algum método se comporta diferente conforme o tipo de veículo?

Exemplo de resposta — Dinâmica 5

- **ABSTRAÇÃO:** Veiculo (placa, modelo, diaria), Cliente (nome, CNH), Locacao (dataInicio, dataFim, calcularTotal()).
- **ENCAPSULAMENTO:** 'saldo do caixa' é privado; 'consultarDisponibilidade()' é público.
- **HERANÇA:** Veiculo → Carro (arCondicionado), Moto (cilindrada), Van (capacidadePassageiros).
- **POLIMORFISMO:** calcularDiaria() → Carro: R\$120; Moto: R\$60; Van: R\$200 — mesmo método, valores diferentes.
- Os 4 princípios trabalham juntos para um sistema organizado, seguro e reutilizável.

"Os quatro princípios — abstração, encapsulamento, herança e polimorfismo — garantem todas as vantagens da utilização do paradigma OO."

Pontos-chave da aula

- Abstração filtra o que é relevante do mundo real para o sistema;
- Encapsulamento protege dados com visibilidade pública/privada;
- Herança permite reutilização: classe base → classes derivadas;
- Polimorfismo: mesmo método, comportamentos diferentes por contexto;
- Os 4 princípios devem ser aplicados do início ao fim de todo projeto OO.